#30/12/2025

Message-Oriented Middleware

Source code here

Modern systems operate in complex environments with multiple programming languages, hardware platforms and operating systems. In order to cope with the demands of such systems, a mechanism called Message-Oriented Middleware or MOM provides a clean method of communication between disparate software entities.

In this article the reader will find the fundamental aspects of this technology together with supporting source code to supplement the information provided.

Author of the article: *Iker Elg. Alzaa*

#MessageOrientedMiddleware #MOM #Integration #OPC

https://github.com/kraftcoding

As software systems continue to be distributed deployments over ever-increasing scales, transcending geographical, organisational, and traditional commercial boundaries, the demands placed upon their communication will increase exponentially.

In these environments, the traditional direct Remote Procedure Call (RPC) mechanisms quickly fail to meet the challenges present.

So, to meet the requirements of this scenario MOM systems have emerged as middleware infrastructure that provides messaging capabilities and infrastructure.

Interaction Models

A client of a MOM system can send messages to, and receive messages from, other clients of the messaging system. Each client connects to one or more servers that act as an intermediary in the sending and receiving of messages.

It uses a model with a peer-to-peer relationship between individual clients; in this model, each peer can send and receive messages to and from other client peers.

Participants in a MOM-based system are not required to block and wait on a message send, they are allowed to continue processing once a message has been sent. This allows the delivery of messages when the sender or receiver is not active or available to respond at the time of execution. Similar to the postal service, messages are delivered to the post office; the postal service then takes responsibility for safe delivery of the message.

This kind of platforms allow flexible cohesive systems to be created.

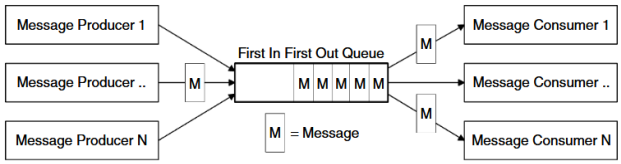
A cohesive system is one that allows changes in one part of a system to occur without the need for changes in other parts of the system. This simplifies the process of building dynamic high-flexible enterprise-class distributed systems.

Commercial implementations provide high scalability with support for tens of thousands of clients, advanced filtering, easy integration into heterogeneous networks, and clustering reliability.

Message Queues

Queues provide the ability to store messages on the platform. So the clients are able to send and receive messages to and from a queue. Queues are central to the implementation of the asynchronous interaction model within this systems.

It support multiple queue types, each with a different purpose. But, the standard queue found in a messaging system is the First-In First-Out (FIFO) queue.



Potentially each application may have its own queue, or applications may share a queue, there is no restriction on the setup.

Messaging Models

Two main message models are commonly available, the point-to-point and publish/subscribe models.

Both of these models are based on the exchange of messages through a channel

(queue). A typical system will utilise a mix of these models to achieve different messaging objectives (in the provided source code we can find examples of these methods).

The point-to-point messaging model provides a straightforward asynchronous exchange of messages between software entities, while the publish/subscribe messaging model, is a very powerful mechanism used to disseminate information between anonymous message consumers and producers.

These one-to-many and many-to-many distribution mechanisms allow a single producer to send a message to one user or potentially hundreds of thousands of consumers.

Hierarchical Channel Namespaces

Hierarchical channels or topics are a destination grouping mechanism in pub/sub messaging model.

This type of structure allows channels to be defined in a hierarchical fashion, so that they may be nested under other channels.

Each sub channel offers a more granular selection of the messages contained in its parent. Clients of hierarchical channels subscribe to the most appropriate level of channel in order to receive the most relevant message.

The relationship between a channel and sub channels allows for super-type subscriptions, where subscriptions that operate on a parent channel/type will also match all subscriptions of descendant channels/types.

Common Services

Message filtering allows a message consumer/receiver to be selective about the messages it receives from a channel. Filtering can operate on a number of different levels. Filters use Boolean logic expressions to declare messages of interest to the client.

On the another hand, transactions provide the ability to group tasks together into a single unit of work.

Message Formats

Depending on the implementation, a number of message formats may be available to the user. Some of the more common message types include Text (including XML),Object, Stream, Hash-Maps, Streaming Multimedia, and so on.